GREEEN tallgrass prairie restoration: Seeded richness ~ Seeding time * Herbicide pre-treatment (NW Station)

MB

2025-10-01

Contents

Preparation **1**

Statistics **2**
Data exploration . . . 2
Means and deviations . . . 2
Graphs of raw data (Step 2, 6, 7) . . . 4
Outliers, zero-inflation, transformations? (Step 1, 3, 4) . . . 7
Check collinearity part 1 (Step 5) . . . 7
Models . . . 7
Model check . . . 8
DHARMa . . . 8
Check collinearity part 2 (Step 5) . . . 15
Model comparison . . . 15
R2 values . . . 15
AICc . . . 16
Predicted values . . . 16
Summary table . . . 16
Forest plot . . . 17
Effect sizes . . . 18

Session info **19**

> **NOTE:** To compare different models, you only have to change the models in the section 'Load models'

Preparation

Protocol of data exploration (Steps 1-8) used from Zuur et al. (2010) Methods Ecol Evol DOI: 10.1111/2041-210X.12577

```
library(here)
library(tidyverse)
library(ggbeeswarm)
library(patchwork)
library(lme4)
library(glmmTMB)
```

```r
library(DHARMa)
library(emmeans)
```

Packages

```r
sites <- read_csv(
  here("data", "processed", "data_processed_sites.csv"),
  col_names = TRUE, na = c("na", "NA", ""), col_types = cols(
    .default = "?",
    id_plot = "f",
    year = "f",
    seeding_time = col_factor(
      levels = c("unseeded", "fall", "spring"), ordered = FALSE
      ),
    herbicide = col_factor(levels = c("0", "1"), ordered = FALSE),
    treatment_id = "f",
    richness_type = "f"
  )
) %>%
  filter(
    site == "NW Station",
    richness_type == "seeded_richness",
    treatment_id %in% c("1", "2", "3", "4")
  ) %>%
  mutate(
    treatment = str_c(seeding_time, herbicide, seeded_pool, sep = "_"),
    treatment = factor(treatment)
  ) %>%
  mutate(y = richness_1qm + richness_25qm)
```

Load data

Statistics

Data exploration

Means and deviations

```r
Rmisc::CI(sites$y, ci = .95)
```

```
##     upper      mean     lower
## 1.3221223 1.0520833 0.7820443
```

```r
median(sites$y)
```

```
## [1] 1
```

```r
sd(sites$y)
```

```
## [1] 1.332744
```

```r
quantile(sites$y, probs = c(0.05, 0.95), na.rm = TRUE)
```

```
##  5% 95%
##   0   4
```

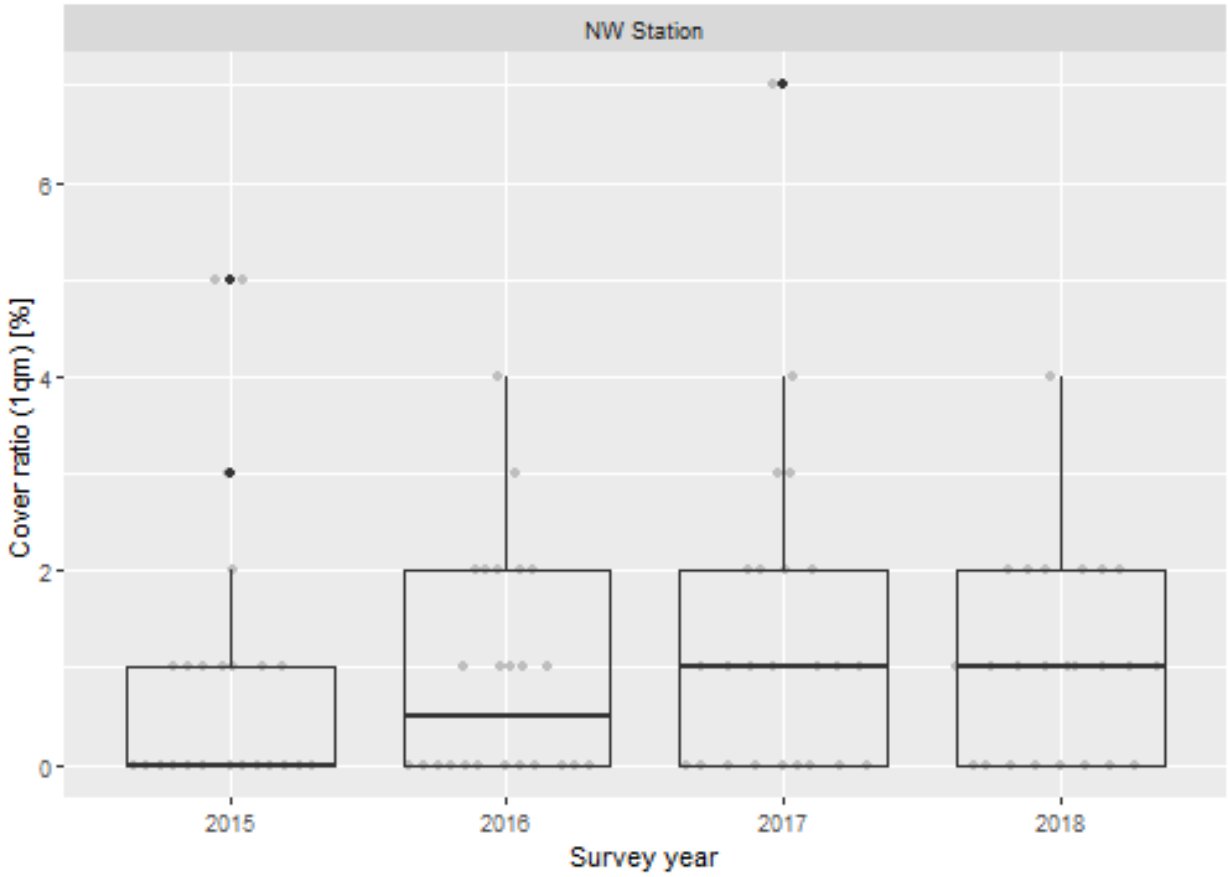
```r
sites %>% count(site, year)
```

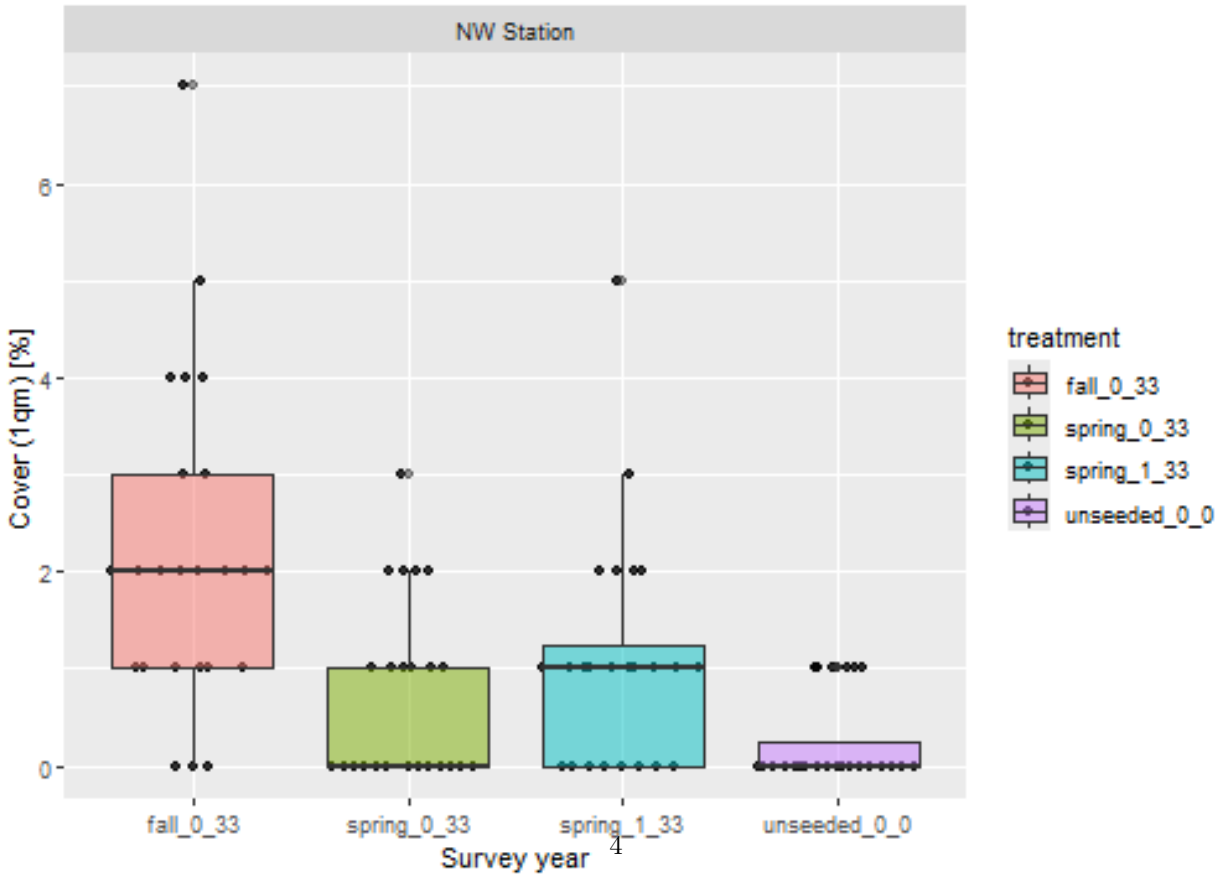
```
## # A tibble: 4 x 3
##   site       year      n
##   <chr>      <fct> <int>
## 1 NW Station 2015     24
## 2 NW Station 2016     24
## 3 NW Station 2017     24
## 4 NW Station 2018     24
```

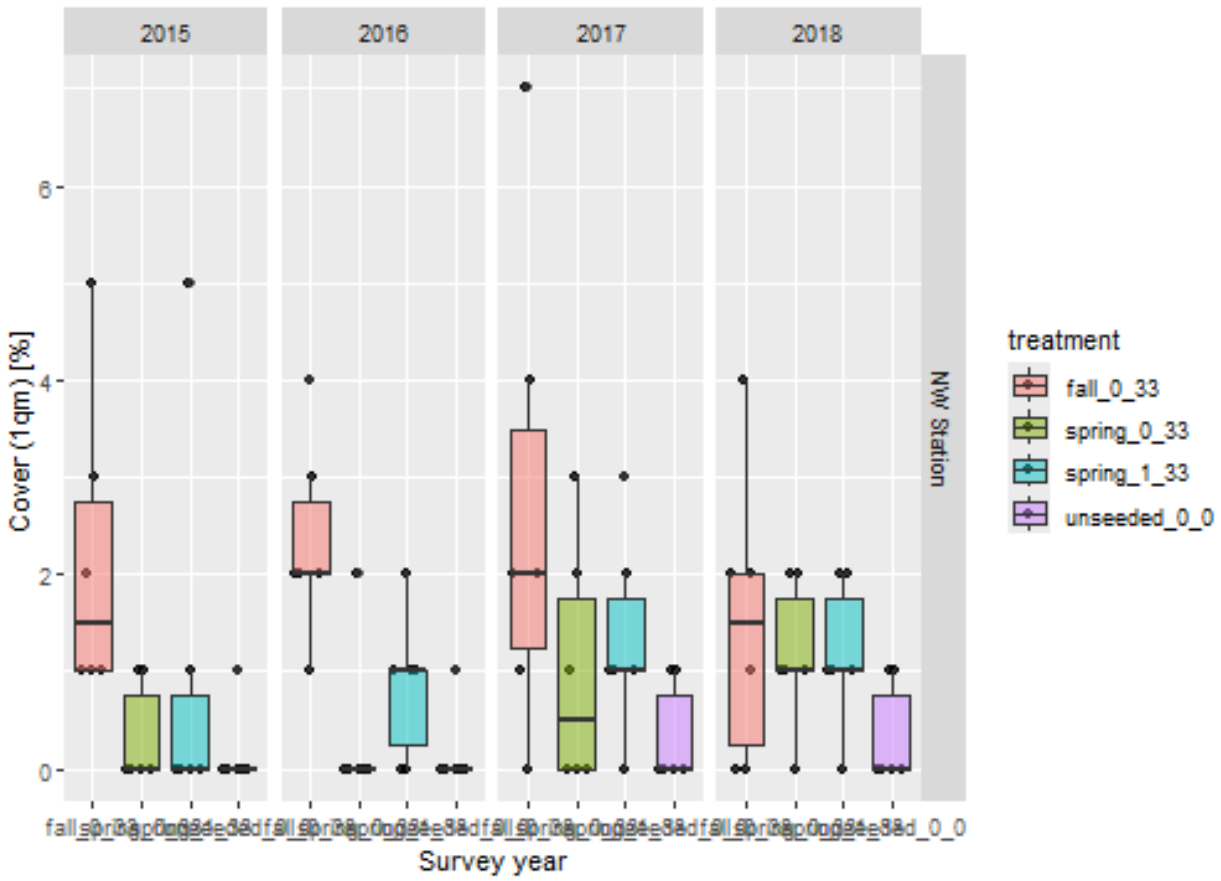
```r
sites %>% count(treatment)
```

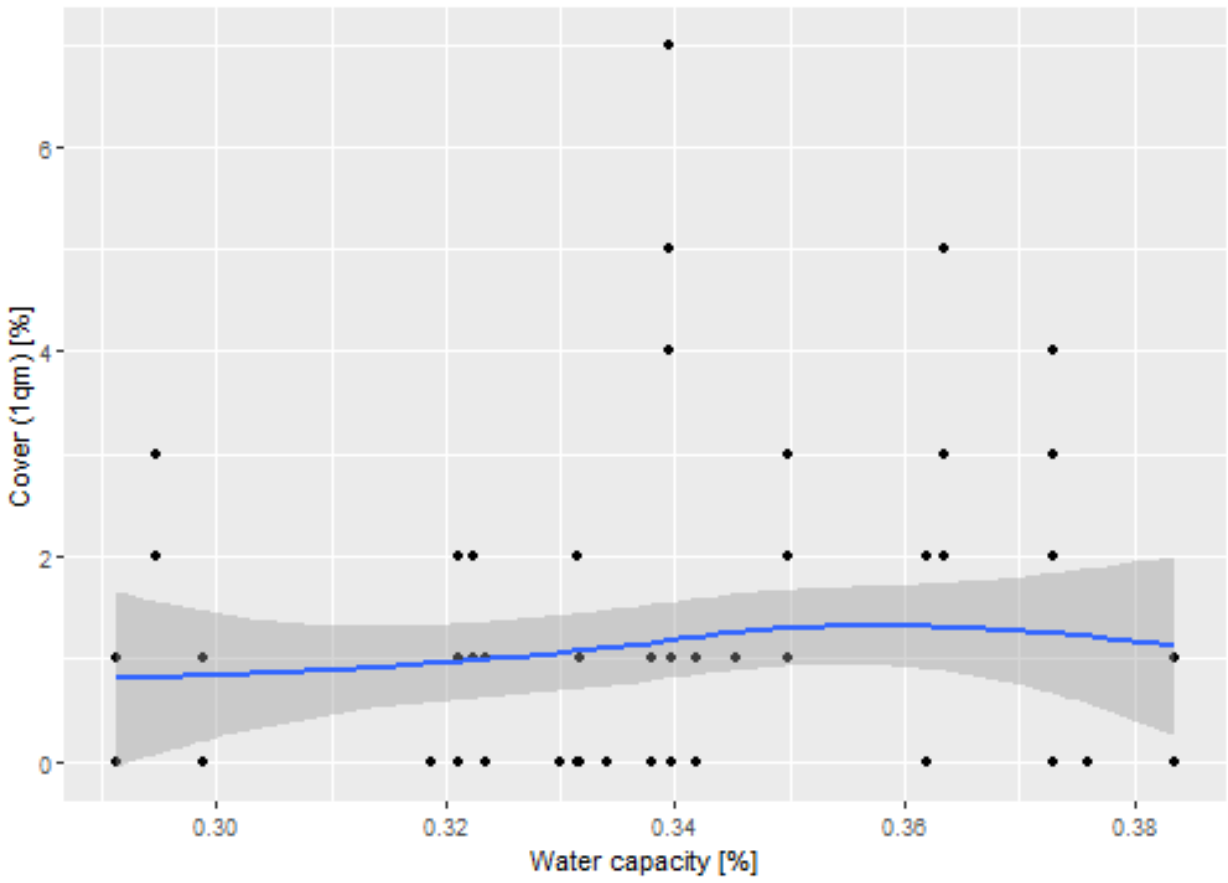
```
## # A tibble: 4 x 2
##   treatment        n
##   <fct>        <int>
## 1 fall_0_33       24
## 2 spring_0_33     24
## 3 spring_1_33     24
## 4 unseeded_0_0    24
```

Graphs of raw data (Step 2, 6, 7)

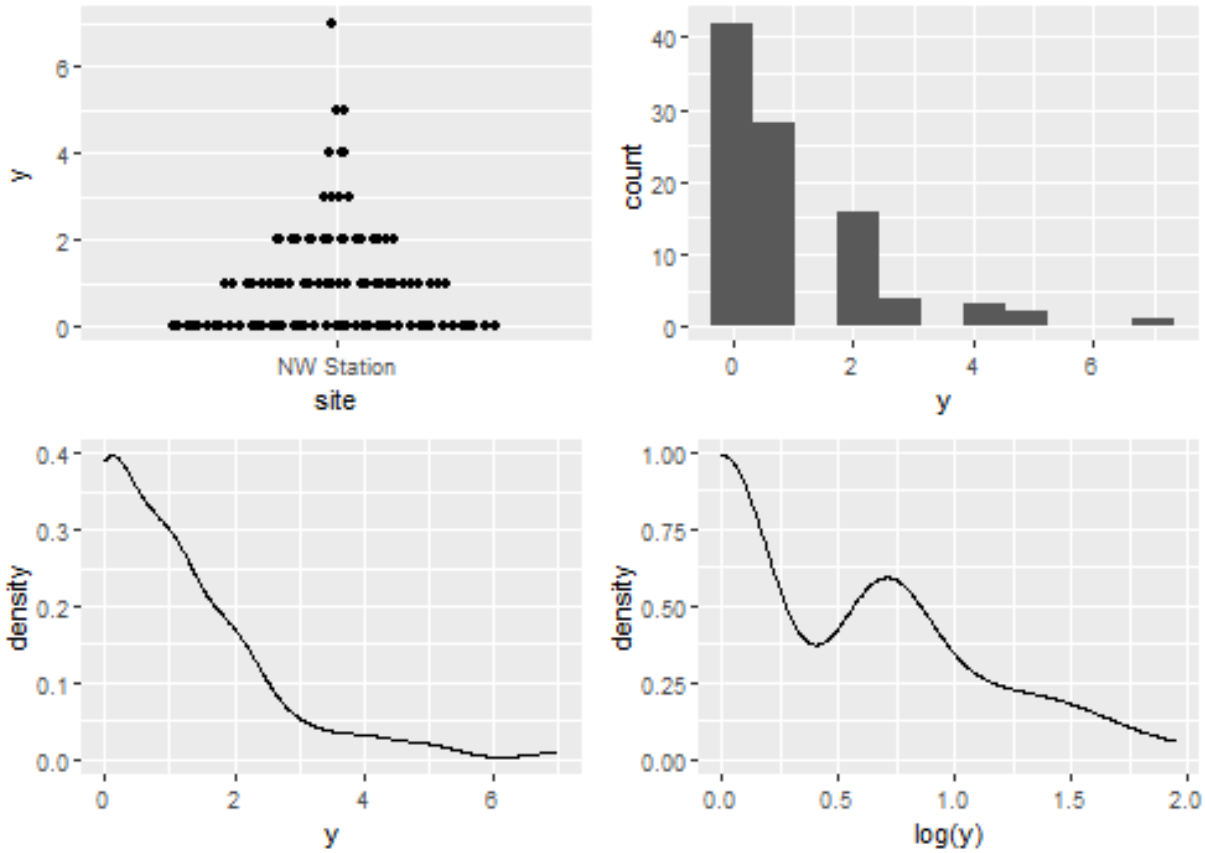





2015
2016
2017
2018
NW Station
Cover (1qm) [%]
0
2
4
6
Survey year
treatment
fall_0_33
spring_0_33
spring_1_33
unseeded_0_0


Cover (1qm) [%]
6
4
2
0
0.30
0.32
0.34
0.36
0.38
Water capacity [%]

Outliers, zero-inflation, transformations? (Step 1, 3, 4)



Check collinearity part 1 (Step 5)

Exclude r > 0.7 Dormann et al. 2013 Ecography DOI: 10.1111/j.1600-0587.2012.07348.x

```
# sites %>%
#   select(where(is.numeric), -y, -starts_with("cwm.")) %>%
#   GGally::ggpairs(
#     lower = list(continuous = "smooth_loess")
#     ) +
#   theme(strip.text = element_text(size = 7))

# -> no continuous variables
```

Models

NOTE: Only here you have to modify the script to compare other models

```
load(file = here("outputs", "models", "model_richness_timing_herbicide_nws_1.Rdata"))
load(file = here("outputs", "models", "model_richness_timing_herbicide_nws_2.Rdata"))
m_1 <- m1
m_2 <- m2
```

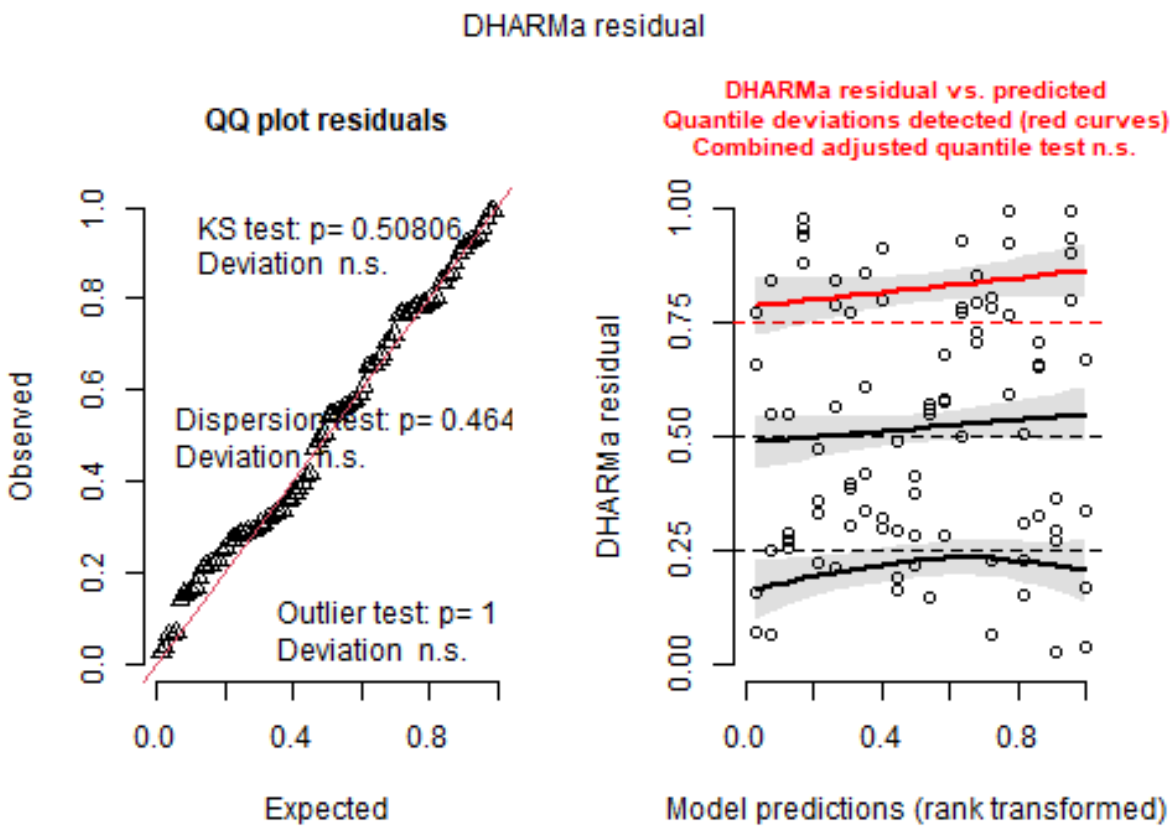
```
m_1@call
## glmer(formula = y ~ treatment + water_cap + (1 | year), data = sites,
##     family = poisson(link = "log"))
```

```r
formula(m_2)
## y ~ treatment * year + water_cap
# formula(m_1)
# m_2@call
```

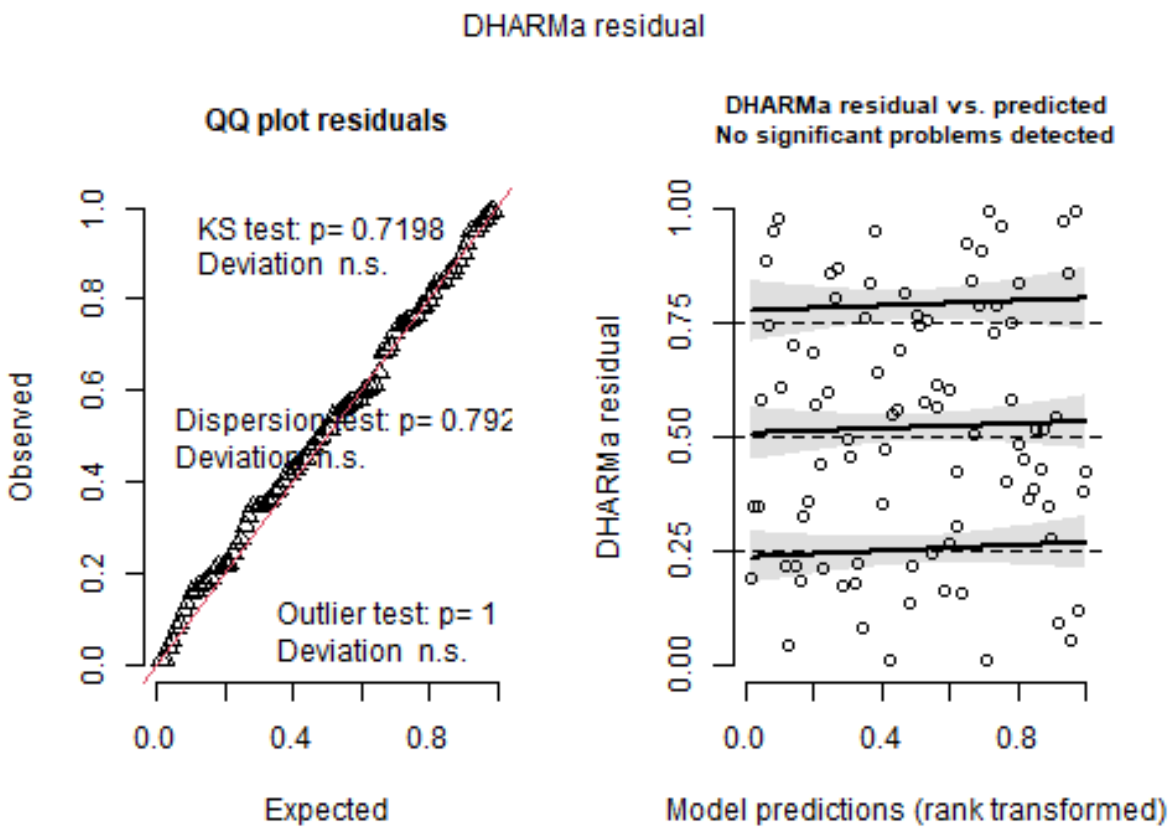
Model check

DHARMa

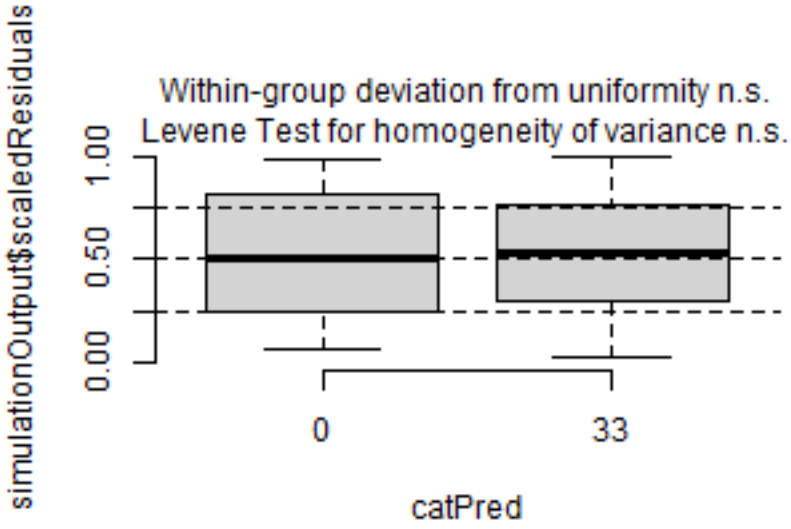
```r
simulation_output_1 <- simulateResiduals(m_1, plot = TRUE)
```




```r
simulation_output_2 <- simulateResiduals(m_2, plot = TRUE)
```

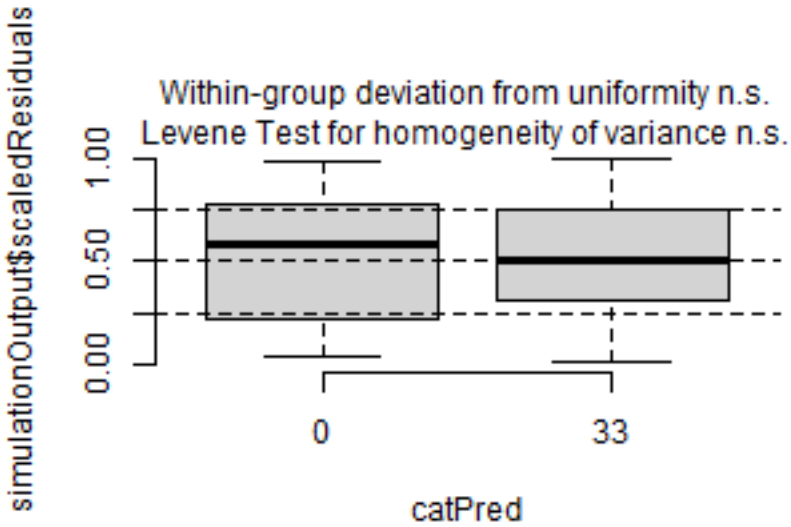



```
plotResiduals(
  simulation_output_1$scaledResiduals,
  factor(sites %>% filter(!(is.na(water_cap))) %>%  pull(seeded_pool))
  )
```

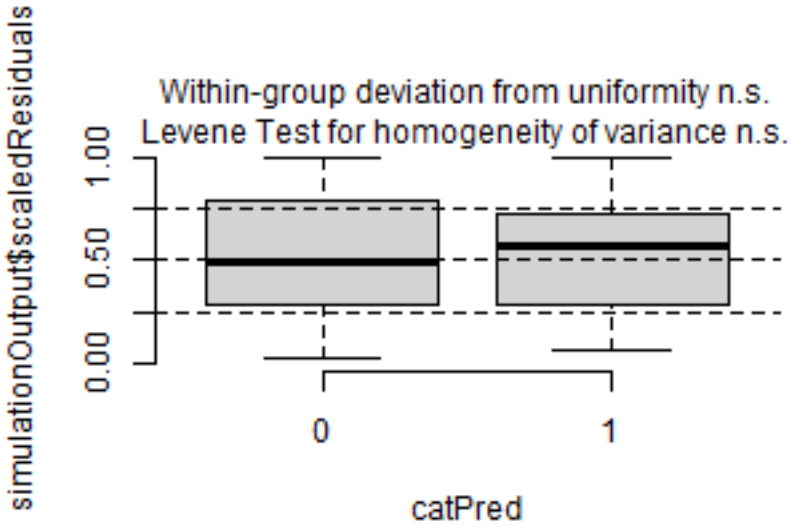



```
plotResiduals(
  simulation_output_2$scaledResiduals,
```

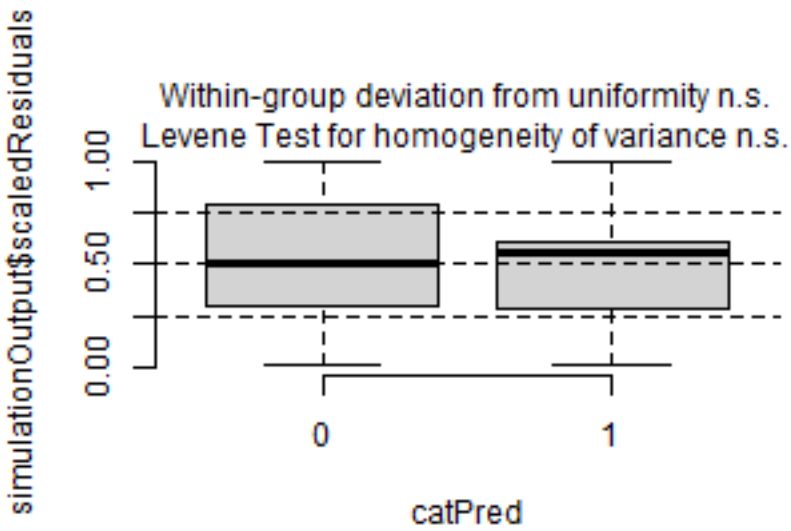
```
  factor(sites %>% filter(!(is.na(water_cap))) %>%  pull(seeded_pool))
  )
```



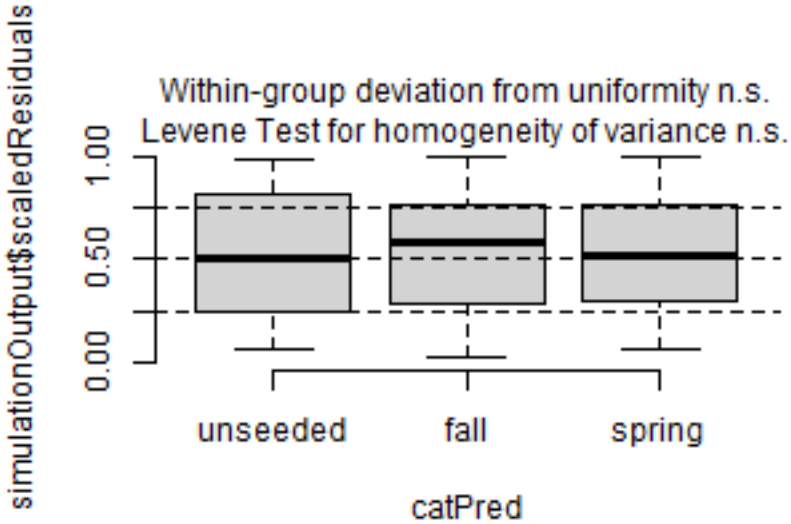

```
plotResiduals(
  simulation_output_1$scaledResiduals,
  factor(sites %>% filter(!(is.na(water_cap))) %>%  pull(herbicide))
  )
```



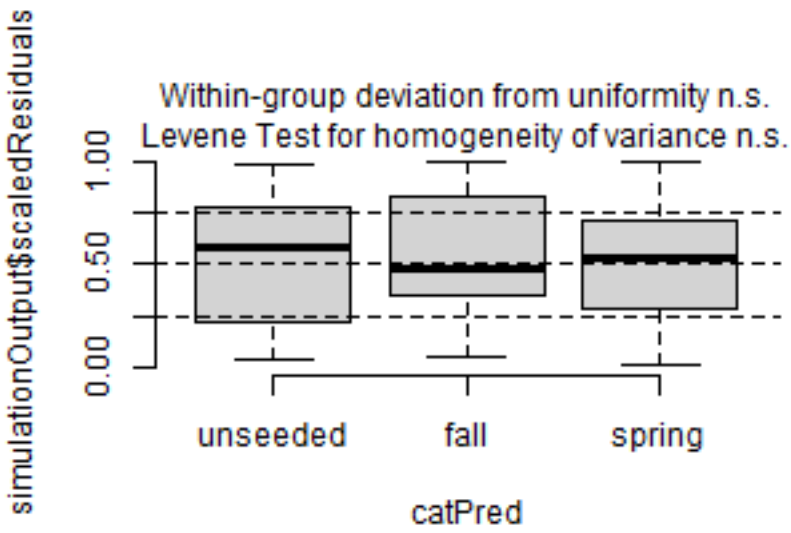

```
plotResiduals(
  simulation_output_2$scaledResiduals,
  factor(sites %>% filter(!(is.na(water_cap))) %>%  pull(herbicide))
  )
```



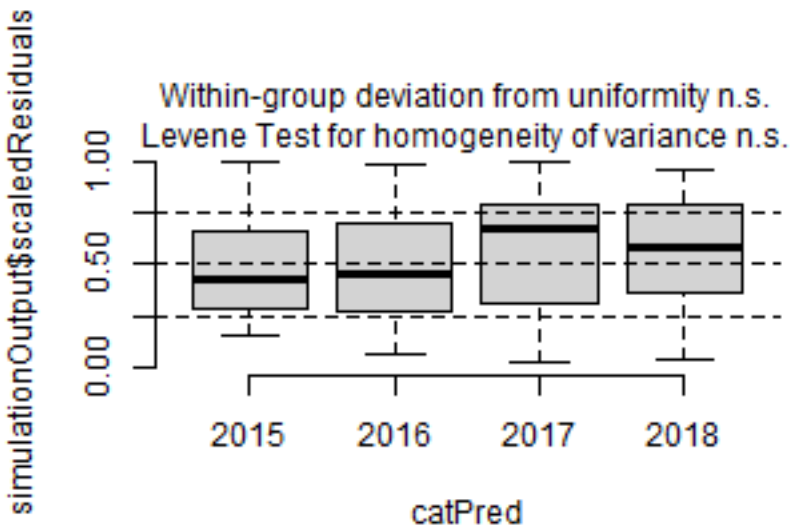

```
plotResiduals(
  simulation_output_1$scaledResiduals,
  factor(sites %>% filter(!(is.na(water_cap))) %>%  pull(seeding_time))
  )
```



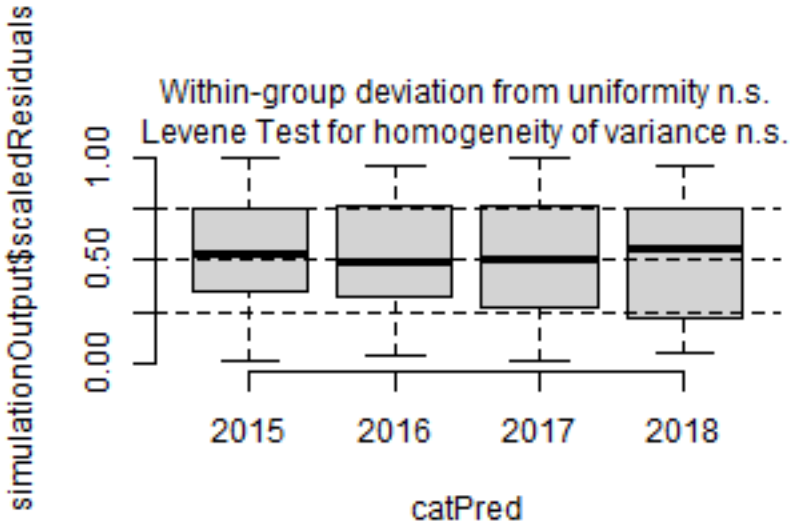

```
plotResiduals(
  simulation_output_2$scaledResiduals,
  factor(sites %>% filter(!(is.na(water_cap))) %>%  pull(seeding_time))
  )
```



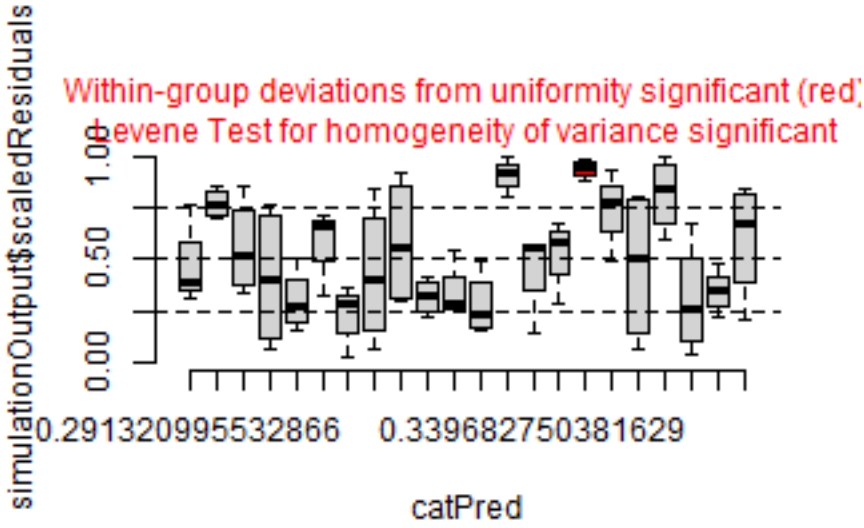

```
plotResiduals(
  simulation_output_1$scaledResiduals,
  factor(sites %>% filter(!(is.na(water_cap))) %>%  pull(year))
  )
```



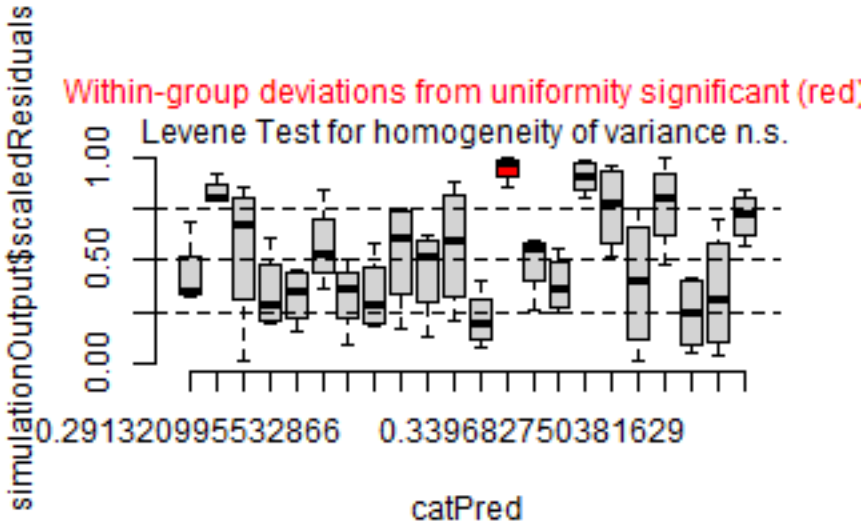

```
plotResiduals(
  simulation_output_2$scaledResiduals,
  factor(sites %>% filter(!(is.na(water_cap))) %>%  pull(year))
  )
```




```
plotResiduals(
  simulation_output_1$scaledResiduals,
  factor(sites %>% filter(!(is.na(water_cap))) %>%  pull(water_cap))
  )
```

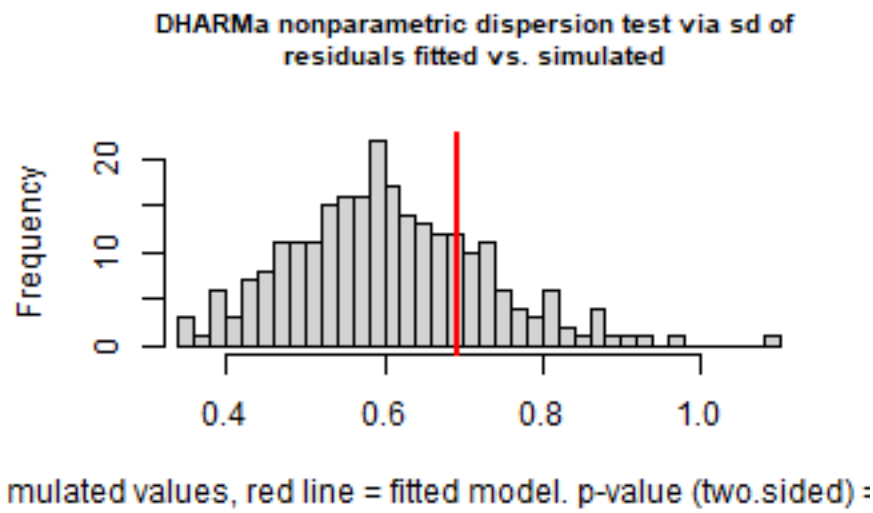



```
plotResiduals(
  simulation_output_2$scaledResiduals,
  factor(sites %>% filter(!(is.na(water_cap))) %>%  pull(water_cap))
  )
```

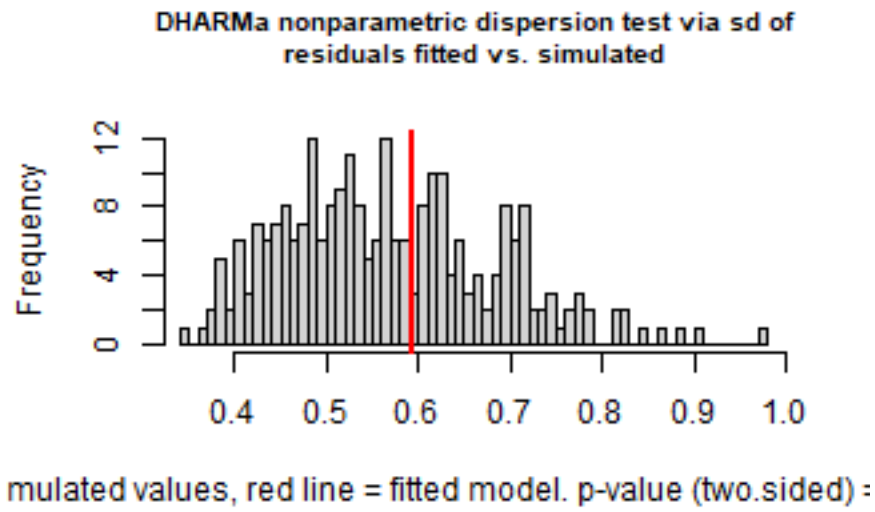


Test for overdispersion

```
testDispersion(simulation_output_1)
```




```
##
##  DHARMa nonparametric dispersion test via sd of residuals fitted vs.
##  simulated
##
## data:  simulationOutput
## dispersion = 1.1428, p-value = 0.464
## alternative hypothesis: two.sided
testDispersion(simulation_output_2)
```




```
##
##  DHARMa nonparametric dispersion test via sd of residuals fitted vs.
##  simulated
##
## data:  simulationOutput
## dispersion = 1.0438, p-value = 0.792
## alternative hypothesis: two.sided
```

Check collinearity part 2 (Step 5)

Remove VIF > 3 or > 10 Zuur et al. 2010 Methods Ecol Evol DOI: 10.1111/j.2041-210X.2009.00001.x

```
car::vif(m_1)
```

```
##               GVIF Df GVIF^(1/(2*Df))
## treatment 1.133911  3        1.021166
## water_cap 1.133911  1        1.064853
```

```
car::vif(m_2)
```

```
## there are higher-order terms (interactions) in this model
## consider setting type = 'predictor'; see ?vif
```

```
##                      GVIF Df GVIF^(1/(2*Df))
## treatment      176.729412  3        2.368926
## year             7.015306  3        1.383591
## water_cap        1.133947  1        1.064870
## treatment:year 783.669119  9        1.448055
```

Model comparison

R2 values

```
MuMIn::r.squaredGLMM(m_1)
## Warning: the null model is only correct if all the variables it uses are identical
## to those used in fitting the original model.
## boundary (singular) fit: see help('isSingular')
##                  R2m       R2c
```

```
## delta     0.4491552 0.4491552
## lognormal 0.5370534 0.5370534
## trigamma  0.3365289 0.3365289
MuMIn::r.squaredGLMM(m_2)
## Warning: the null model is only correct if all the variables it uses are identical
## to those used in fitting the original model.
##                 R2m       R2c
## delta     0.4955775 0.4955775
## lognormal 0.5829462 0.5829462
## trigamma  0.3793266 0.3793266
```

AICc

Use AICc and not AIC since ratio n/K < 40 Burnahm & Anderson 2002 p. 66 ISBN: 978-0-387-95364-9

```
MuMIn::AICc(m_1, m_2) %>%
  arrange(AICc)
##     df     AICc
## m_1  6 226.7741
## m_2 17 248.1065
```

Predicted values

Summary table

```
car::Anova(m_1, type = 3)
```

```
## Analysis of Deviance Table (Type III Wald chisquare tests)
##
## Response: y
##               Chisq Df Pr(>Chisq)
## (Intercept)  5.8478  1   0.015596 *
## treatment   38.4682  3   2.25e-08 ***
## water_cap    8.7997  1   0.003013 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
summary(m_1)
```

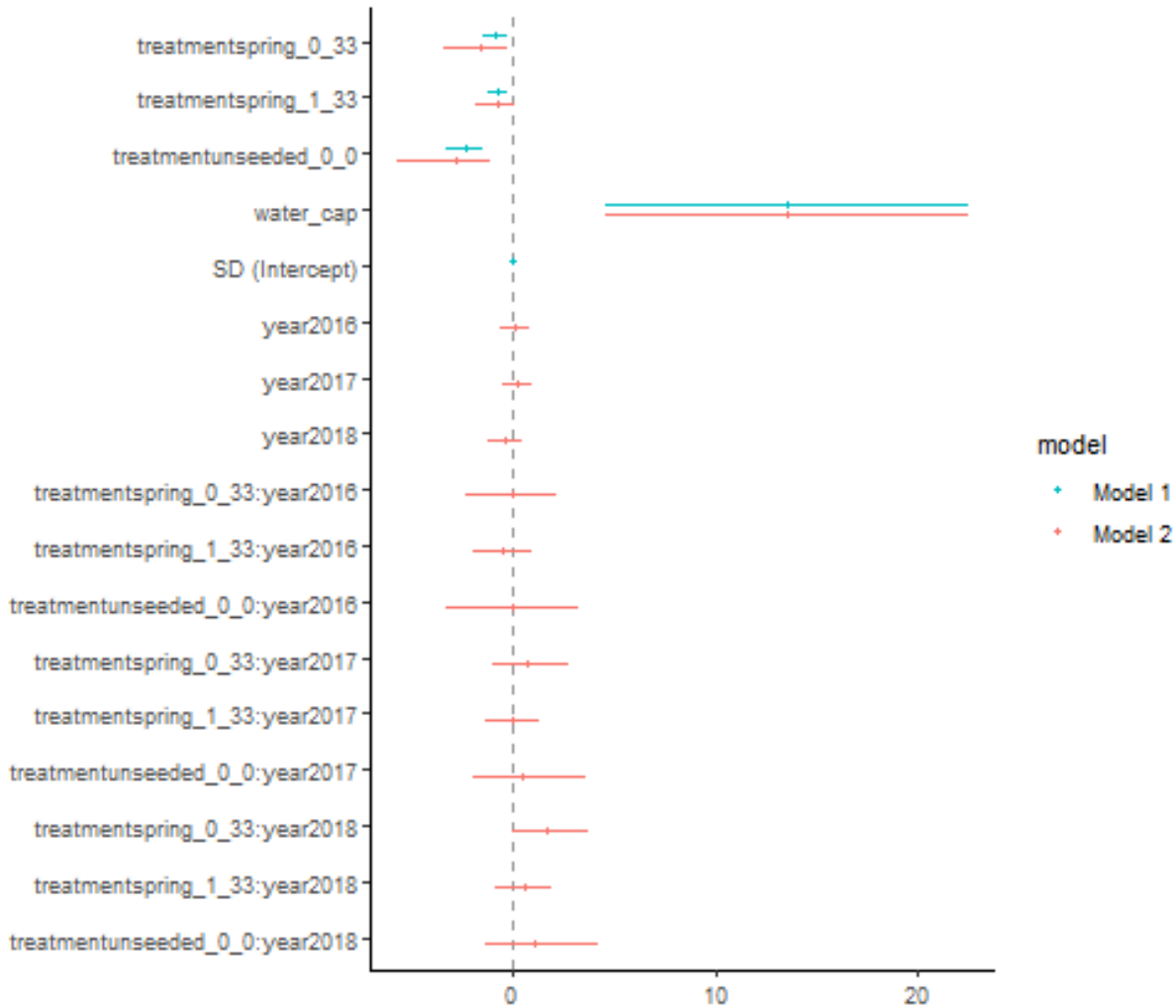
```
## Generalized linear mixed model fit by maximum likelihood (Laplace
##   Approximation) [glmerMod]
##  Family: poisson  ( log )
## Formula: y ~ treatment + water_cap + (1 | year)
##    Data: sites
##
##       AIC       BIC    logLik -2*log(L)  df.resid
##     225.7     240.6    -106.9     213.7        82
##
## Scaled residuals:
##     Min      1Q  Median      3Q     Max
## -1.9287 -0.6562 -0.3864  0.5993  3.0190
##
## Random effects:
##  Groups Name        Variance Std.Dev.
##  year   (Intercept) 0        0
```

```
## Number of obs: 88, groups:  year, 4
##
## Fixed effects:
##                       Estimate Std. Error z value Pr(>|z|)
## (Intercept)            -3.7418     1.5473  -2.418  0.01560 *
## treatmentspring_0_33   -0.8912     0.2886  -3.087  0.00202 **
## treatmentspring_1_33   -0.8051     0.2535  -3.175  0.00150 **
## treatmentunseeded_0_0  -2.4152     0.4396  -5.494 3.92e-08 ***
## water_cap              13.5572     4.5702   2.966  0.00301 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation of Fixed Effects:
##             (Intr) t_0_33 t_1_33 tr_0_0
## trtmnt_0_33 -0.180
## trtmnt_1_33  0.179  0.231
## trtmntn_0_0  0.166  0.125  0.217
## water_cap   -0.996  0.138 -0.229 -0.195
## optimizer (Nelder_Mead) convergence code: 0 (OK)
## boundary (singular) fit: see help('isSingular')
```

Forest plot

```r
dotwhisker::dwplot(
  list(m_1, m_2),
  ci = 0.95,
  show_intercept = FALSE,
  vline = geom_vline(xintercept = 0, colour = "grey60", linetype = 2)) +
  theme_classic()
```

```
## Your model may suffer from singularity (see see `?lme4::isSingular` and
##   `?performance::check_singularity`).
##   Some of the standard errors and confidence intervals of the random
##   effects parameters are probably not meaningful!
##   You may try to impose a prior on the random effects parameters, e.g.
##   using the glmmTMB package.
```



Effect sizes

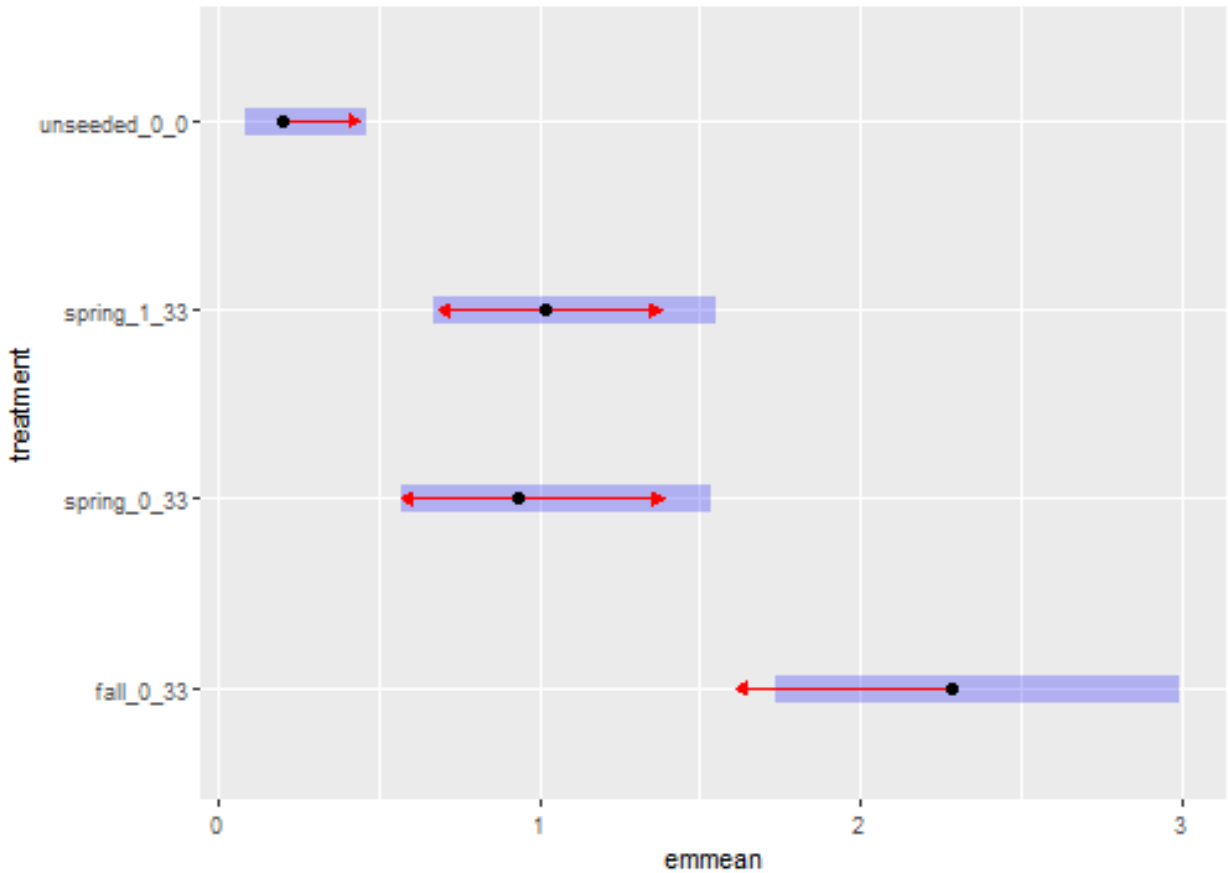
Effect sizes of chosen model just to get exact values of means etc. if necessary.

```
(emm <- emmeans(
  m_1,
  revpairwise ~ treatment,
  type = "response"
  ))
```

```
## $emmeans
##  treatment     rate     SE  df asymp.LCL asymp.UCL
##  fall_0_33    2.281 0.3160 Inf    1.7378     2.993
##  spring_0_33  0.935 0.2370 Inf    0.5699     1.536
##  spring_1_33  1.020 0.2170 Inf    0.6719     1.547
##  unseeded_0_0 0.204 0.0851 Inf    0.0899     0.462
##
## Confidence level used: 0.95
## Intervals are back-transformed from the log scale
##
```

```
## $contrasts
##  contrast                     ratio     SE  df null z.ratio p.value
##  spring_0_33 / fall_0_33    0.4102 0.1180 Inf    1  -3.087  0.0109
##  spring_1_33 / fall_0_33    0.4470 0.1130 Inf    1  -3.175  0.0081
##  spring_1_33 / spring_0_33  1.0899 0.3680 Inf    1   0.255  0.9942
##  unseeded_0_0 / fall_0_33   0.0894 0.0393 Inf    1  -5.494  <.0001
##  unseeded_0_0 / spring_0_33 0.2178 0.1080 Inf    1  -3.080  0.0111
##  unseeded_0_0 / spring_1_33 0.1999 0.0914 Inf    1  -3.521  0.0024
##
## P value adjustment: tukey method for comparing a family of 4 estimates
## Tests are performed on the log scale
```

```r
plot(emm, comparison = TRUE)
```



Session info

```
## R version 4.5.1 (2025-06-13 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26100)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=German_Germany.utf8  LC_CTYPE=German_Germany.utf8
```

```
## [3] LC_MONETARY=German_Germany.utf8 LC_NUMERIC=C
## [5] LC_TIME=German_Germany.utf8
##
## time zone: Europe/Berlin
## tzcode source: internal
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
##  [1] emmeans_1.11.1   DHARMa_0.4.7     glmmTMB_1.1.12   lme4_1.1-37
##  [5] Matrix_1.7-3     patchwork_1.3.1  ggbeeswarm_0.7.2 lubridate_1.9.4
##  [9] forcats_1.0.0    stringr_1.5.1    dplyr_1.1.4      purrr_1.1.0
## [13] readr_2.1.5      tidyr_1.3.1      tibble_3.3.0     ggplot2_3.5.2
## [17] tidyverse_2.0.0  here_1.0.1
##
## loaded via a namespace (and not attached):
##  [1] mnormt_2.1.1           Rdpack_2.6.4           gridExtra_2.3
##  [4] sandwich_3.1-1         rlang_1.1.6            magrittr_2.0.3
##  [7] compiler_4.5.1         mgcv_1.9-3             vctrs_0.6.5
## [10] quadprog_1.5-8         pkgconfig_2.0.3        crayon_1.5.3
## [13] fastmap_1.2.0          backports_1.5.0        labeling_0.4.3
## [16] pbivnorm_0.6.0         utf8_1.2.6             ggstance_0.3.7
## [19] promises_1.3.3         rmarkdown_2.29         tzdb_0.5.0
## [22] nloptr_2.2.1           bit_4.6.0              xfun_0.52
## [25] later_1.4.2            lavaan_0.6-19          parallel_4.5.1
## [28] R6_2.6.1               gap.datasets_0.0.6     stringi_1.8.7
## [31] qgam_2.0.0             RColorBrewer_1.1-3     car_3.1-3
## [34] boot_1.3-31            numDeriv_2016.8-1.1    estimability_1.5.1
## [37] Rcpp_1.1.0             iterators_1.0.14       knitr_1.50
## [40] zoo_1.8-14             parameters_0.27.0      httpuv_1.6.16
## [43] splines_4.5.1          timechange_0.3.0       tidyselect_1.2.1
## [46] rstudioapi_0.17.1      abind_1.4-8            yaml_2.3.10
## [49] MuMIn_1.48.11          doParallel_1.0.17      TMB_1.9.17
## [52] codetools_0.2-20       nonnest2_0.5-8         lattice_0.22-7
## [55] plyr_1.8.9             shiny_1.11.1           withr_3.0.2
## [58] bayestestR_0.16.1      coda_0.19-4.1          evaluate_1.0.4
## [61] marginaleffects_0.28.0 CompQuadForm_1.4.4     pillar_1.11.0
## [64] gap_1.6                carData_3.0-5          foreach_1.5.2
## [67] stats4_4.5.1           reformulas_0.4.1       insight_1.3.1
## [70] generics_0.1.4         vroom_1.6.5            rprojroot_2.1.1
## [73] hms_1.1.3              scales_1.4.0           minqa_1.2.8
## [76] xtable_1.8-4           glue_1.8.0             tools_4.5.1
## [79] data.table_1.17.8      mvtnorm_1.3-3          grid_4.5.1
## [82] rbibutils_2.3          datawizard_1.1.0       nlme_3.1-168
## [85] Rmisc_1.5.1            performance_0.15.0     beeswarm_0.4.0
## [88] vipor_0.4.7            Formula_1.2-5          cli_3.6.5
## [91] gtable_0.3.6           digest_0.6.37          farver_2.1.2
## [94] htmltools_0.5.8.1      lifecycle_1.0.4        mime_0.13
## [97] bit64_4.6.0-1          dotwhisker_0.8.4       MASS_7.3-65
```